

Real-world (a real approval pause) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `approval_pause_practice.py`: this is the exact `interrupt()` pattern the Build Task needs at its final gate — pause, show a human the draft and its sources, wait for a real decision. **If not:** the Build Task's approval step would be the first place you ever used `interrupt()` for a real reason, with no smaller version to trust.

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# approval_pause_practice.py
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import MemorySaver

def approval_node(state):
    decision = interrupt({"draft": state["draft"], "sources": state["sources"]})
    return {"approved": decision == "approved"}

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "thread-1"}}

# first call -- pauses at approval_node
result = graph.invoke({"task": "What is our refund policy?"}, config)
print("Paused, waiting for approval:", result)

# simulate the human saying yes
final = graph.invoke(Command(resume="approved"), config)
print("Final result:", final)
```

This works, and correctly pauses and resumes. It doesn't check what happens on rejection — it only tests the approved path.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints, an explicit prompt, and testing both outcomes

```
# approval_pause_practice.py
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import MemorySaver

def approval_node(state: dict) -> dict:
    decision = interrupt({
        "draft": state["draft"],
        "sources": state["sources"],
        "prompt": "Approve this answer before it's sent? (approved/rejected)",
    })
    return {"approved": decision == "approved"}

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

def run_with_approval(task: str, decision: str) -> dict:
    config = {"configurable": {"thread_id": f"thread-{task[:8]}"}}
    paused = graph.invoke({"task": task}, config)
    print("Paused for approval:", paused)
    final = graph.invoke(Command(resume=decision), config)
    return final

approved_run = run_with_approval("What is our refund policy?", "approved")
print("Approved run result:", approved_run)

rejected_run = run_with_approval("What is our refund policy?", "rejected")
print("Rejected run result:", rejected_run)
```

Difference from Basic: full type hints. The interrupt payload includes an explicit prompt string, so a real reviewer (not just you, testing) knows what's being asked. And this version tests *both* the approved and rejected paths, not just the happy path — which is the only way to actually confirm

your graph reacts correctly to a real "no." This version still trusts the resume value blindly and keeps no record of who decided or when — that's what Approach 2 adds.

Approach 2 — a structured resume payload, recording who and when

Story: a bare resume string tells the graph *what* was decided but nothing about *who* or *when* — the moment anything goes wrong, "who approved this?" is a real question someone asks, and a bare string has no answer. **If not:** the Build Task's approval step would have no way to show a later reviewer who actually signed off, or when.

```
# approval_pause_practice.py
from datetime import datetime, timezone
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import MemorySaver

def approval_node(state: dict) -> dict:
    paused_at = datetime.now(timezone.utc).isoformat()
    decision = interrupt({
        "draft": state["draft"],
        "sources": state["sources"],
        "paused_at": paused_at,
        "prompt": "Approve this answer before it's sent?",
    })
    return {
        "approved": decision["decision"] == "approved",
        "approved_by": decision.get("by"),
        "approved_at": decision.get("at"),
    }

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "thread-audit-1"}}

graph.invoke({"task": "What is our refund policy?"}, config)

final = graph.invoke(
    Command(resume={
        "decision": "approved",
        "by": "jane@example.com",
        "at": datetime.now(timezone.utc).isoformat(),
    }),
    config,
```

```
)
print(f"Approved by {final['approved_by']} at {final['approved_at']}")
```

The node's own logic (`decision["decision"] == "approved"`) is barely different from Approach 1 — the real change is that the resume value now carries an identity and a timestamp alongside the yes/no, and both get saved into state instead of discarded the moment the decision is made.

Approach 3 — the same audit trail, plus a staleness check before trusting an old resume

Story: recording who and when only helps if something actually looks at it — this adds the check itself: before treating a resumed decision as valid, confirm the pause hasn't sat open so long that the original search results might be outdated. **If not:** a month-old approval, made against documents that have since changed, would get treated exactly like one made two minutes ago.

```
# approval_pause_practice.py
from datetime import datetime, timedelta, timezone
from langgraph.types import interrupt, Command

MAX_APPROVAL_AGE_HOURS = 168 # 1 week

def approval_node(state: dict) -> dict:
    paused_at = datetime.now(timezone.utc).isoformat()
    decision = interrupt({
        "draft": state["draft"],
        "sources": state["sources"],
        "paused_at": paused_at,
    })

    if is_stale(decision["paused_at"] if "paused_at" in decision else paused_at):
        return {
            "approved": False,
            "stale": True,
            "answer": "This approval is stale -- the underlying documents may have
                changed since this was searched. Please re-run the search."
        }

    return {
        "approved": decision["decision"] == "approved",
        "approved_by": decision.get("by"),
        "approved_at": decision.get("at"),
        "stale": False,
    }
```

```
def is_stale(paused_at: str, max_age_hours: int = MAX_APPROVAL_AGE_HOURS) -> bool:
    paused_time = datetime.fromisoformat(paused_at)
    age = datetime.now(timezone.utc) - paused_time
    return age > timedelta(hours=max_age_hours)

# test: a decision that arrives "on time"
recent_pause = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
assert is_stale(recent_pause) is False

# test: a decision that arrives long after the pause
old_pause = (datetime.now(timezone.utc) - timedelta(days=30)).isoformat()
assert is_stale(old_pause) is True
print("Staleness check behaves correctly for both a recent and an old pause.")
```

`is_stale` doesn't stop the resume from happening — LangGraph doesn't know or care how old an interrupt is — it's a check your own node makes *after* resuming, so a month-old approval doesn't get treated identically to one made two minutes ago.

Difference from Approach 1, and between Approaches 2/3: Approach 1's resume value is a bare string — the graph knows *what* was decided but nothing about who or when, and would treat a decision made a month ago exactly like one made a minute ago. Approach 2 fixes the "who and when" gap with a structured resume payload. Approach 3 uses that same timestamp to answer a question Approach 2 records but doesn't act on: is this decision still trustworthy, or was it made against grounding that's since gone stale?

Which one should you actually write? Always test both the approved and rejected paths, like Approach 1 does — a graph that only handles "approved" correctly but silently does the risky thing anyway on rejection is a real bug you'd only catch by testing rejection specifically. Add Approach 2's structured resume payload the moment more than one person can approve things, or the moment "who approved this" is a question that could reasonably come up later — which, for anything touching real actions, is almost always. Add Approach 3's staleness check once your paused threads can realistically sit open for hours or days, not seconds — a fast-moving internal tool with reviewers watching a queue may never need it; a support ticket approved once a week by a manager on rotation needs it from day one.